Given network traffic captured in Packet Capture (PCAP) files, the objective is to identify and analyze network events, detect abnormal behavior or communication failures, and predict appropriate remedial measures to resolve the identified issues.

The system will extract relevant features from packet-level traffic, identify network events such as connection failures, retransmissions, DNS resolution failures, and abnormal TCP behavior, and determine the likely causes of these events. Based on the observed network patterns, the system will recommend suitable troubleshooting actions to help resolve the identified issues.

Show more

PCAP-Based Network Event Detection & Remediation System

What You're Building

A pipeline that reads PCAP files → extracts features → classifies network events (using ML) → recommends fixes. Think of it as an automated Wireshark analyst.

Prerequisites

Languages & Tools

- Python 3.10+
- Wireshark / `tshark` (CLI version) — for PCAP parsing
- `pyshark` or `scapy` — Python PCAP libraries
- Jupyter Notebook — for exploration

Python Libraries

- `scapy`, `pyshark` — packet parsing
- `pandas`, `numpy` — feature engineering
- `scikit-learn`, `xgboost` — ML models
- `matplotlib`, `seaborn`, `plotly` — visualization
- `flask` or `fastapi` — for the final API/UI

Datasets (Public)

- CAIDA — real traffic traces
- PCAP-ATTACK — attack traffic
- Wireshark Sample Captures — various anomalies
- UNSW-NB15 — labeled network events

Implementation Phases

Phase 1 — Data Ingestion & Parsing (Week 1–2)

Goal: Read raw PCAP files and convert them into structured tabular data.

Steps:

1. Install `tshark` and `scapy`
2. Write a parser using `pyshark` to read each packet
3. Extract raw fields: `src_ip`, `dst_ip`, `src_port`, `dst_port`, `protocol`, `timestamp`, `packet_length`, `TCP flags`, `TTL`, `DNS query/response`, `HTTP status`
4. Store as CSV/Parquet per PCAP file
5. Handle large files in streaming/chunked mode

Key Output: A structured DataFrame per PCAP file

Phase 2 — Feature Engineering (Week 2–3)

Goal: Transform raw packets into meaningful flow-level and session-level features.

Steps:

1. **Flow aggregation** — group packets by 5-tuple (`src_ip`, `dst_ip`, `src_port`, `dst_port`, `protocol`) within a time window
2. Extract features per flow:
   - `packet_count`, `byte_count`, `avg_inter-arrival_time`
   - `tcp_syn_count`, `tcp_rst_count`, `tcp_fin_count`, `retransmission_count`
   - `dns_query_count`, `dns_nxdomain_count`
   - `connection_duration`, `zero_window_count`
   - `avg_ttl`, `ttl_variance`
3. Compute **ratios** — SYN/SYN-ACK ratio (detects SYN flood), RST/total ratio (detects port scans)
4. Label time windows with sliding window technique

Key Output: Feature matrix ready for ML

Phase 3 — Event Labeling & Dataset Preparation (Week 3–4)

Goal: Create a labeled dataset of network events.

Event Classes to Detect:

Label	Trigger Pattern
Connection Failure	SYN with no SYN-ACK
Retransmission Storm	High retransmit ratio
DNS Failure	NXDOMAIN or no response
Port Scan	Many SYNs to different ports, no data
SYN Flood	Massive SYN with no ACK
Slow Network	High RTT, zero-window packets
Normal	None of the above

Steps:

1. Write rule-based labeling functions using the feature matrix
2. Cross-validate labels against known-labeled datasets (UNSW-NB15)
3. Handle class imbalance using SMOTE or class weights
4. Split: 70% train / 15% val / 15% test

Phase 4 — ML Model Development (Week 4–6)

Goal: Train classifiers to predict event type from features.

Steps:

1. Start with a **baseline** — Random Forest (interpretable, robust)
2. Try **XGBoost** — usually best on tabular network data
3. Try **LSTM** if you model packet sequences (time-series approach)
4. Evaluation metrics: F1-score (macro), confusion matrix, precision-recall per class
5. Use **SHAP values** to explain which features drive each prediction

Model for Remediation:

- Either a second classifier trained on (event → remediation action)
- Or a rule-based mapper on top of the event classifier (simpler, explainable)

Remediation Action Classes:

- Check firewall rules
- Investigate DNS server
- Rate-limit source IP
- Check server resource exhaustion
- Inspect routing table
- No action needed

Phase 5 — Pipeline Integration (Week 6–7)

Goal: Connect parsing → feature extraction → model inference into one pipeline.

Steps:

1. Build a `PCAPAnalyzer` class encapsulating all phases
2. Input: path to `.pcap` file
3. Output: list of detected events with confidence scores + recommended actions
4. Add a summary report generator (JSON + human-readable text)
5. Write unit tests for each module

Phase 6 — UI / API (Week 7–8)

Goal: Make it usable by network admins.

Steps:

1. Build a REST API with FastAPI — endpoint: `POST /analyze` (upload PCAP, get events)
2. Build a simple dashboard with Plotly Dash or Streamlit:
   - Upload PCAP
   - See timeline of events
   - Per-flow drill-down
   - Remediation recommendations panel
3. (Optional) Real-time mode using `scapy` live capture → feed into same pipeline

---

Phase 7 — Evaluation & Documentation (Week 8–9)

Goal: Validate the system end-to-end and document everything.

Steps:

1. Test on 3–4 diverse PCAP datasets you haven't trained on
2. Compare ML classifier vs pure rule-based baseline
3. Measure end-to-end latency (time from PCAP upload to report)

4. Write final report covering: architecture, results, limitations, future work

---

Suggested Tech Stack Summary

```
PCAP File
   ↓
pyshark / scapy (Phase 1)
   ↓
pandas feature matrix (Phase 2)
   ↓
XGBoost classifier (Phase 4)
   ↓
Rule-mapped remediation (Phase 4)
   ↓
FastAPI + Streamlit dashboard (Phase 6)
```

Quick Start for Phase 1

python

```python
import pyshark

cap = pyshark.FileCapture('sample.pcap')
for pkt in cap:
    print(pkt.highest_layer, pkt.length)
```

Want me to dive deep into any specific phase — like the feature engineering code, the ML model setup, or the labeling logic?

> the main exphasis is on understanding the various network events happening like spatial temporal errors and all the ML part is secondary

Got it — the core of this project is **deep understanding of what's actually happening in the network traffic**, not just throwing ML at it. ML is just the final classifier on top of well-understood patterns.

Let me reframe the whole thing around **network event understanding.**

The Real Core: Understanding Network Events from PCAPs

A PCAP is essentially a **timestamped log of every packet** that crossed the wire. Your job is to read those packets and reconstruct *what went wrong and when*.

Categories of Network Events You'll Analyze

---

1. TCP-Level Events

TCP is stateful — it has a handshake, data transfer, and teardown. Any deviation is an event.

a) Connection Failure (Refused)

- Client sends `SYN`
- Server responds `RST/ACK` (port closed) or sends nothing
- In PCAP: you see a lone SYN with no SYN-ACK following it within a timeout window
- Cause: server down, firewall blocking, wrong port

b) SYN Timeout / Half-Open Connection

- Client sends `SYN`, retransmits it 3–6 times, then gives up
- In PCAP: repeated SYNs to the same `dst_ip:dst_port` with exponential back-off intervals (1s, 2s, 4s, 8s...)
- Cause: server unreachable, packet loss on path, firewall silently dropping

c) Retransmission

- A packet is sent, no ACK received, so TCP resends it
- In PCAP: same sequence number appears twice with a time gap
- Types:
  - **Fast Retransmit** — triggered by 3 duplicate ACKs (short gap, ~ms)
  - **RTO Retransmit** — triggered by timeout (longer gap, ~1s+)
- High retransmission rate = congestion or lossy link

d) Duplicate ACKs

- Receiver keeps ACKing the last good byte it got
- In PCAP: same ACK number repeated 3+ times from the same host
- Precedes fast retransmit — signals a gap in the receive buffer

e) Zero Window

- Receiver's buffer is full → advertises `win=0`
- Sender must stop sending until `Window Update` arrives
- In PCAP: `TCP ZeroWindow` packets, then `TCP Window Update`
- Cause: slow application consuming data, memory pressure on server

f) RST Flood / Unexpected RST

- Connection abruptly killed by a RST
- In PCAP: RST mid-stream (not during handshake)
- Cause: firewall injection, NAT timeout, server crash, port scan detection

g) Out-of-Order Packets

- Packets arrive in wrong sequence number order
- In PCAP: Wireshark flags these as `TCP Out-of-Order`
- Cause: multiple paths (ECMP routing), packet reordering in switches

---

2. Spatial-Temporal Patterns (This is Key)

"Spatial" = *where* in the network topology the events occur
"Temporal" = *when* they cluster in time

Spatial Analysis:

- Is the same `dst_ip` seeing connection failures from many different source IPs? → Server is down or overloaded
- Are failures concentrated on one subnet? → Routing issue or switch problem
- Is only one specific `src_ip` causing RSTs everywhere? → Misconfigured host or attacker

Temporal Analysis:

- **Burst patterns** — 50 retransmissions in 200ms, then normal → transient congestion
- **Periodic patterns** — connection failures every 30 seconds exactly → keepalive misconfiguration or scheduled task failing
- **Escalating patterns** — retransmissions increasing over time → progressive congestion buildup
- **Correlated events** — DNS failure followed immediately by connection failure to a new IP → DNS is lying or slow

How to measure temporally:

- Inter-arrival time between events of the same type

- Event rate per time window (events/second in a sliding window)
- Time-to-first-event after a triggering packet

---

3. DNS Events

DNS sits under everything — a DNS failure cascades into connection failures.

a) NXDOMAIN (Non-Existent Domain)

- Client queries a hostname, DNS server says it doesn't exist
- In PCAP: DNS response with `RCODE = 3`
- Cause: typo in hostname, domain expired, DNS poisoning

b) DNS Timeout / No Response

- Client sends query, no answer arrives within ~5 seconds
- In PCAP: query packet with no matching response (match by `transaction ID`)
- Cause: DNS server unreachable, UDP blocked by firewall

c) DNS Retransmission

- Same query resent (same transaction ID) multiple times
- In PCAP: identical query packets spaced ~2–5 seconds apart

d) DNS-to-TCP Fallback

- Response too large for UDP (>512 bytes) → DNS falls back to TCP
- In PCAP: UDP query with `TC` (truncated) flag set, followed by TCP connection to port 53
- Cause: DNSSEC records, large TXT records

e) Slow DNS Resolution

- Time between DNS query and response > 500ms
- This is a direct latency contributor — all connections to that hostname stall waiting

---

4. Application-Layer Events

HTTP:

- `4xx` responses — client error (bad request, auth failure, not found)
- `5xx` responses — server error
- Slow `Time to First Byte (TTFB)` — time from request to first byte of response
- Connection reuse failures — `Connection: close` when keep-alive expected

TLS/SSL:

- Handshake failures — `Alert` messages (`certificate_expired`, `handshake_failure`)
- In PCAP: TLS Alert record right after Client Hello or after certificate exchange
- Cause: expired cert, cipher mismatch, SNI mismatch

---

5. ICMP Events

- `ICMP Destination Unreachable` — router can't forward packet (host down, port unreachable, network unreachable)
- `ICMP Time Exceeded` — TTL hit zero (routing loop or traceroute artifact)
- `ICMP Fragmentation Needed` — packet too large, and DF bit is set → Path MTU Discovery failure
    - This one is subtle: causes silent TCP stalls on connections over VPNs

---

6. Behavioral / Anomaly Events

These are patterns rather than single packets:

Port Scan:

- One source IP → many destination ports on the same host
- In PCAP: SYN to port 22, SYN to port 23, SYN to port 80... in rapid succession
- Most get RST back (closed ports)

SYN Flood (DoS):

- Massive SYNs from many IPs to one destination
- Server's half-open connection table fills up
- In PCAP: thousands of SYNs, very few completing the handshake

Bandwidth Saturation:

- Sudden spike in bytes/second
- Competing flows start showing retransmissions and zero windows simultaneously

---

How to Extract All This from a PCAP

```
PCAP file
   ↓
Parse each packet → (timestamp, src_ip, dst_ip, src_port, dst_port, protocol,
flags, seq, ack, len, TTL)
   ↓
Reconstruct TCP streams → group by 4-tuple + match seq/ack numbers
   ↓
Apply event detectors (rule-based functions) per stream:
   - retransmission_detector(stream)
   - zero_window_detector(stream)
   - syn_timeout_detector(stream)
   - dns_failure_detector(dns_flows)
   ↓
Tag each event with: (event_type, timestamp, src, dst, severity)
   ↓
Spatial-temporal aggregation:
   - cluster by IP / subnet / time window
   - compute event rates, correlations
   ↓
Root cause inference:
   - "DNS failure on X followed by connection failures to X's IPs" → DNS
issue
   - "Retransmissions correlated with zero-windows on same server" → server
overload
```

Event Detection Logic (Rule-Based, Not ML)

This is your main engine:

python

```python
def detect_syn_timeout(stream):
    syns = [p for p in stream if 'SYN' in p.flags and 'ACK' not in p.flags]
    syn_acks = [p for p in stream if 'SYN' in p.flags and 'ACK' in p.flags]
    if len(syns) >= 2 and len(syn_acks) == 0:
        gaps = [syns[i+1].time - syns[i].time for i in range(len(syns)-1)]
        if all(gaps[i+1] > gaps[i] * 1.5 for i in range(len(gaps)-1)):
            return Event('SYN_TIMEOUT', exponential_backoff=True)
```

Each event type gets one such detector. These are precise, explainable, and don't need training data.

Where ML Fits (Secondary, as You Said)

Once you have clean event labels from your rule-based detectors, ML helps with:

1. **Ambiguous cases** — when multiple events co-occur and you need to rank the most likely root cause
2. **Novelty detection** — catching patterns your rules don't cover
3. **Remediation ranking** — given this combination of events, which fix has worked historically

But the rule-based detectors are the ground truth. ML refines on top of that.

Suggested Reading to Go Deeper

- *TCP/IP Illustrated Vol. 1* — Stevens (the bible for understanding these events)
- Wireshark's own display filters documentation — each filter name maps directly to an event type
- RFC 793 (TCP), RFC 1035 (DNS) — the specs tell you exactly what's normal vs abnormal

---

Want me to go deeper on any specific event type — say, the spatial-temporal correlation logic, or how to reconstruct TCP streams from raw packets?

